

A Cutting-Edge Parallel Solver for Scalable Power Grid Analysis Using Nested Domain Decomposition

Jianfei Song¹, Xiaoyu Yang², Zhou Jin¹, Cheng Zhuo^{1*}

¹Zhejiang University, Hangzhou, China

²China University of Petroleum-Beijing, Beijing, China

*Corresponding author: czhuo@zju.edu.cn

Abstract—As transistor scaling approaches sub-5nm technologies, power distribution networks (PDNs) in integrated circuits have grown increasingly complex, with billions to trillions of nodes. Simultaneously, reduced noise margins and increased power density necessitate more accurate and efficient power grid analysis. Traditional methods for solving large-scale PDNs, especially those requiring the solution of sparse linear systems, face significant challenges due to high computational costs. Although domain decomposition methods (DDM) allow for efficient parallel computation, the size of the dense global Schur complement grows excessively large as the number of partitions increases, limiting scalability and imposing substantial computational burdens. This paper introduces an efficient parallel nested domain decomposition solver that incorporates a parallel Schur complement computation strategy and intermediate Schur complement to address these challenges. Experimental results demonstrate that by introducing an intermediate Schur complement, the size of the global Schur complement is significantly reduced, achieving an average 1.70× speedup in computation, which results in a 1.30× speedup for the entire solver compared to the conventional DDM parallel solver.

I. INTRODUCTION

As Moore's law drives transistor scaling to its limits, power distribution networks (PDNs) in integrated circuits have grown increasingly complex, encompassing billions to even trillions of nodes in sub-5nm technologies [1], [2]. Meanwhile, reduced noise margins, now as low as tens of millivolts due to lower supply voltages, and rising power density [3], [4] have made accurate and efficient power grid analysis more critical than ever. However, the computational demands of analyzing large-scale PDNs significantly hinder design efficiency and iteration speed. Consequently, enhancing the efficiency of power grid analysis methods has become a pivotal research focus.

Power grid analysis typically involves solving large-scale symmetric positive definite (SPD) linear systems, which can be addressed using direct or iterative solvers. Direct methods, such as Cholesky and LU decomposition [5], [6], provide exact solutions but are limited by high memory requirements, making them unsuitable for scalability on a single machine. Iterative methods, including Krylov subspace techniques [7], algebraic multigrid approaches [8], and graph spectral sparsification approaches [9], [10] offer better memory efficiency but often face stability and convergence challenges if preconditioners are not selected appropriately.

The rise of parallel computing technology has brought domain decomposition methods (DDM) to the forefront as a

powerful approach for large-scale power grid analysis [11], [12]. These methods, which rely on graph partitioning, enable the division of PDN into smaller subdomains that can be processed in parallel, significantly improving computational efficiency. However, DDM relies on the Schur complement matrix. **Forming and solving the dense global Schur complement matrix can be even more demanding than directly solving the original system** [13], [14]. This challenge involves two main aspects:

- Calculating the local Schur complement for each subdomain is inherently complex and resource-demanding due to intricate interdependencies.
- While current computational resources are capable of handling many subdomains, excessive partitioning increases the size of the dense global Schur complement, making this large matrix challenging to parallelize and ultimately diminishing computational efficiency.

To overcome the above challenges, this paper introduces **an efficient parallel nested domain decomposition solver** specifically for power grid analysis, focusing on accelerating the formation of the local Schur complement, reducing the size of the global Schur complement matrix and addressing the limited parallelizability at the top level to ensure better scalability and resource utilization. This solver integrates multiple parallelization strategies to tackle the challenges associated with solving large-scale sparse linear systems, optimizing Schur complement matrix computation, and providing a comprehensive solution to enhance computational efficiency. The main contributions are as follows:

- **A parallel method for forming the Schur complement.** In this method, the nodes to be eliminated are divided into segments and processed in parallel, utilizing DDM to partition and solve in parallel blocks.
- **An intermediate Schur complement matrix to reduce the size of the dense global Schur complement.** This intermediate matrix can be naturally solved in parallel, further optimizing the overall process and improving computation and resource utilization.
- **A new solver that exploits the above optimizations to achieve higher scalability.** This solver overcomes the scalability limitations of conventional DDM solver, which struggles to achieve significant speedup as the number of partitions increases beyond a certain threshold.

Experimental results demonstrate that by introducing an intermediate Schur complement, the overall size of the global Schur complement is significantly reduced, effectively overcoming the limitations of traditional DDM. The intermediate Schur complement is well-suited for parallel computation, enabling task distribution to the intermediate layer and achieving a 1.70× speedup, which leads to our solver outperforming the conventional DDM solver with a 1.30× speedup. Furthermore, it achieves an 8.44× speedup over the CKTSO parallel solver and an 8.36× speedup over the CHOLMOD-based serial direct solver when utilizing 32 threads.

II. BACKGROUND AND MOTIVATION

A. Introduction to DDM for Power Grid Analysis

DDM is a parallel method for solving large linear systems of the form $Gv = b$, particularly effective in PDN analysis [11], [12]. It partitions the PDN into subdomains by linking the coefficient matrix to an undirected graph and applying the Schur complement to enhance computational efficiency. Each subdomain contains internal nodes (fully within the subdomain) and interface nodes (shared between subdomains), forming a block sparse matrix that supports efficient parallel processing. The structure of the block sparse matrix is as follows:

$$\begin{bmatrix} A_1 & B_1 & O & O & \cdots & O & O \\ B_1^T & D_1 & O & E_{1,2} & \cdots & O & E_{1,m} \\ O & O & A_2 & B_2 & \cdots & O & O \\ O & E_{1,2}^T & B_2^T & D_2 & \cdots & O & E_{2,m} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ O & O & O & O & \cdots & A_m & B_m \\ O & E_{1,m}^T & O & E_{2,m}^T & \cdots & B_m^T & D_m \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ x_2 \\ y_2 \\ \vdots \\ x_m \\ y_m \end{bmatrix} = \begin{bmatrix} f_1 \\ g_1 \\ f_2 \\ g_2 \\ \vdots \\ f_m \\ g_m \end{bmatrix} \quad (1)$$

where A_i , B_i , and D_i represent the connections within the i -th subdomain, and $E_{i,j}$ represents the interactions between the i -th and j -th subdomains.

DDM works by first eliminating internal nodes using the Schur complement, as in Eq. (2)-(4), reducing the problem to a smaller system involving only interface nodes, which is given by Eq. (5). This reduction makes the problem more suitable for parallel computation by decreasing both the size and complexity of the system. Once the interface nodes are solved, the internal nodes can then be calculated using Eq. (6).

$$\begin{bmatrix} P_i A_i P_i^T & P_i B_i \\ B_i^T P_i^T & D_i \end{bmatrix} = \begin{bmatrix} L_{i,11} & O \\ L_{i,21} & L_{i,22} \end{bmatrix} \begin{bmatrix} L_{i,11}^T & L_{i,21}^T \\ O & L_{i,22}^T \end{bmatrix} \quad (2)$$

$$\widetilde{D}_i = D_i - B_i^T A_i^{-1} B_i = L_{i,22} L_{i,22}^T \quad (3)$$

$$\widetilde{g}_i = g_i - B_i^T A_i^{-1} f_i = g_i - L_{i,21} L_{i,11}^{-1} P_i f_i \quad (4)$$

$$\begin{bmatrix} \widetilde{D}_1 & E_{1,2} & \cdots & E_{1,m} \\ E_{1,2}^T & \widetilde{D}_2 & \cdots & E_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ E_{1,m}^T & E_{2,m}^T & \cdots & \widetilde{D}_m \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} \widetilde{g}_1 \\ \widetilde{g}_2 \\ \vdots \\ \widetilde{g}_m \end{bmatrix} \quad (5)$$

$$A_i x_i = f_i - B_i y_i \quad (6)$$

B. Motivation

For conventional DDM, challenges remain in managing the cost of constructing and solving the dense global Schur complement and handling subdomain dependencies. In large-scale parallel computing, circuits are divided into small blocks, with the effects varying by partition count:

- **Fewer partitions:** Fewer partitions lead to longer local Schur complement calculations. *Larger sub-blocks require more time to compute*, limiting the advantages of parallel computing.
- **Too many partitions:** Numerous partitions can make top-level computation time significant, *potentially dominating the entire process*. The increased overhead for managing the global Schur complement reduces the benefits of parallel computing.

TABLE I: Breakdown time comparison for DDM solver.

Thread	2		4		8		16		32	
Case	T_{loc}	T_{top}	T_{loc}	T_{top}	T_{loc}	T_{top}	T_{loc}	T_{top}	T_{loc}	T_{top}
ibmpg8	10.5	0.01	4.03	0.40	2.53	1.65	1.88	3.62	1.38	2.95
thupg1	40.1	0.48	24.1	3.60	8.16	7.96	5.32	13.9	3.89	14.6
thupg2	82.6	0.87	53.1	6.90	25.3	15.7	12.5	22.3	7.95	25.6
thupg3	101	1.16	63.2	9.60	41.8	21.1	17.7	30.0	11.2	29.6
thupg4	154	2.11	91.2	11.3	51.2	25.9	23.5	46.1	16.5	55.7
thupg5	216	3.11	129	31.0	72.8	42.1	45.3	73.2	23.4	100

Table I shows the relationship between top-level computation time and the number of partitions. **With fewer partitions**, the time for local Schur complement calculations, T_{loc} , becomes too long. **With more partitions**, the top-level computation time, T_{top} , becomes dominant. This occurs because the global Schur complement size becomes too large and cannot be efficiently parallelized. Thus, traditional DDM has faced scalability challenges, often struggling to achieve speedup when the number of partitions increases beyond a certain point, as solving the dense global Schur complement tends to dominate the computation time. However, **modern hardware supports a significantly higher number of processes and threads, offering opportunities for enhanced parallelism.**

III. PROPOSED METHOD

This section presents our parallel computing framework for scalable power grid analysis. We first introduce a parallel method for computing the Schur complement, which is then integrated to form a nested DDM approach. This nested structure enhances scalability and computational efficiency, making it ideal for analyzing large-scale power grid systems.

A. Parallel Schur Complement Computation

Forming the Schur complement is closely related to Gaussian elimination. This process simplifies the network by eliminating certain nodes and reducing conductances to equivalent values at the selected ports. By applying Gaussian elimination to the conductance matrix, electrical connections between

nodes are combined into effective conductances at key ports, resulting in a simplified version of the system that preserves its essential electrical behavior.

The DDM also facilitates forming the Schur complement. In this approach, we first identify selected ports and non-port nodes, then apply a graph partitioning tool like METIS [15] to classify nodes. To achieve balanced computation, higher weights are assigned to selected ports, allowing an even distribution across blocks. Afterward, non-port nodes are further divided into internal and interface nodes within each subdomain. This categorization aids in the elimination process and organizes the system into a block-structured format, which is optimized for parallel computation.

Within each subdomain, internal nodes, along with their associated interface nodes and selected ports, are initially extracted to facilitate efficient parallel processing. Subsequently, the internal nodes are eliminated parallelly, producing a streamlined structure for further computations. Finally, interface nodes are removed, yielding the Schur complement corresponding to the selected ports. To clarify this process, Fig. 1 uses edge-based partitioning to depict the steps involved.

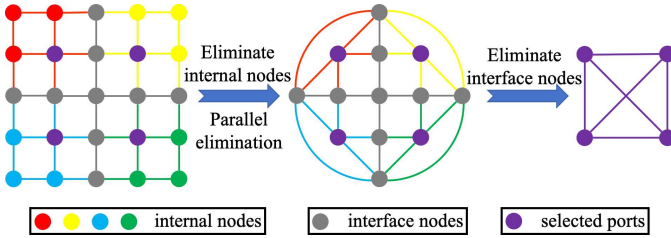


Fig. 1: Flow of parallel Schur complement computation.

This parallel Schur complement approach is not only effective for power grid analysis but can also be applied in scenarios where full-stack PDN simulation is unnecessary. For instance, in machine learning-based methods that predict supply noise [16]–[18]), selected ports can represent locations of worst-case noise. Our proposed parallel Schur complement method enables efficient computation of the noise at these critical locations.

B. Nested DDM for Power Grid Analysis

In conventional DDM for power grid analysis, the system is initially divided into m subdomains. As the number of subdomains increases (i.e., m becomes very large), the size of the global Schur complement matrix grows significantly, resulting in a dense matrix that is difficult to solve in parallel. To mitigate this, we apply a nested decomposition approach to the PDN, including the following steps.

Partitioning to construct nested DDM form matrix. The original matrix G can be transformed into the nested DDM form through a two-step graph partitioning process. First, the nodes corresponding to the entire matrix are partitioned to form the outer subdomains. Then the outer internal nodes within each outer subdomain are subdivided into inner interface nodes and inner internal nodes. This process classifies nodes into three types: *outer interface nodes* (e.g., gray nodes

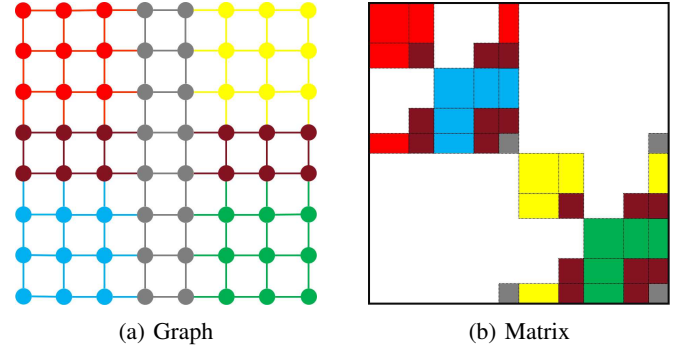


Fig. 2: Graph partitioning to nested DDM: node colors in the graph (left) correspond to the sub-matrix block colors in the matrix (right).

in Fig. 2), *inner interface nodes* (e.g., brown nodes in Fig. 2), and *inner internal nodes* (e.g., other colors in Fig. 2). After applying the corresponding permutation matrix, the transformed matrix takes the final form presented in Eq. (7).

For inner nodes, the matrices $A_{11}, \dots, A_{pq}$ represent the inner internal nodes within the $p \times q$ inner subdomains, while $D_{11}, \dots, D_{pq}$ correspond to the inner interface nodes. The matrix B_{ij} captures the interactions between the inner internal nodes and the inner interface nodes in the ij -th inner subdomain, while $E_{ij,st}$ describes the couplings between the inner interface nodes of the ij -th and st -th inner subdomains. The vectors f_{ij} and g_{ij} denote the right-hand side values associated with the inner internal nodes and inner interface nodes of the ij -th inner subdomain, respectively.

For outer interface nodes and their couplings, the matrix R_i represents the interactions among the outer interface nodes within the i -th outer subdomain. The matrix P_{ij} captures the coupling between the outer interface nodes of the i -th outer subdomain and the inner internal nodes of the ij -th inner subdomain, while Q_{ij} represents the coupling between the outer interface nodes of the i -th outer subdomain and the inner interface nodes of the ij -th inner subdomain. Additionally, $F_{i,j}$ characterizes the couplings between the outer interface nodes of the i -th and j -th outer subdomains. Finally, h_i refers to the right-hand side values associated with the outer interface nodes of the i -th outer subdomain.

Next step is to eliminate inner internal nodes.

1. Extract submatrices A_{ij} , B_{ij} , D_{ij} , P_{ij} , Q_{ij} , R_i , along with the right-hand sides f_{ij} , g_{ij} , and h_i parallelly for inner internal nodes, inner interface nodes, and related outer interface nodes, as shown in Eq. (7) within each inner subdomain.

2. Form matrices parallelly for inner interface nodes:

$$\widetilde{D}_{ij} = D_{ij} - B_{ij}^T A_{ij}^{-1} B_{ij} \quad (8)$$

3. Form matrices parallelly for outer interface nodes:

$$\widetilde{R}_i = R_i - \sum_{j=1}^q P_{ij}^T A_{ij}^{-1} P_{ij} \quad (9)$$

4. Form coupling matrices parallelly between inner and outer interface nodes:

$$\widetilde{Q}_{ij} = Q_{ij} - B_{ij}^T A_{ij}^{-1} P_{ij} \quad (10)$$

$$\begin{bmatrix}
A_{11} & B_{11} & \cdots & O & O & P_{11} & \cdots & O & O & \cdots & O & O & O \\
B_{11}^T & D_{11} & \cdots & O & E_{11,1q} & Q_{11} & \cdots & O & O & \cdots & O & O & O \\
\vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\
O & O & \cdots & A_{1q} & B_{1q} & P_{1q} & \cdots & O & O & \cdots & O & O & O \\
O & E_{11,1q}^T & \cdots & B_{1q}^T & D_{1q} & Q_{1q} & \cdots & O & O & \cdots & O & O & O \\
P_{11}^T & Q_{11}^T & \cdots & P_{1q}^T & Q_{1q}^T & R_1 & \cdots & O & O & \cdots & O & O & F_{1,p} \\
\vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\
O & O & \cdots & O & O & O & \cdots & A_{p1}^T & B_{p1} & \cdots & O & O & P_{p1} \\
O & O & \cdots & O & O & O & \cdots & B_{p1}^T & D_{p1} & \cdots & O & E_{p1,pq} & Q_{p1} \\
\vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\
O & O & \cdots & O & O & O & \cdots & O & O & \cdots & A_{pq} & B_{pq} & P_{pq} \\
O & O & \cdots & O & O & O & \cdots & O & E_{p1,pq}^T & \cdots & B_{pq}^T & D_{pq} & Q_{pq} \\
O & O & \cdots & O & O & F_{1,p}^T & \cdots & P_{p1}^T & Q_{p1}^T & \cdots & P_{pq}^T & Q_{pq}^T & R_p
\end{bmatrix}
\begin{bmatrix}
x_{11} \\ y_{11} \\ \vdots \\ x_{1q} \\ y_{1q} \\ z_1 \\ \vdots \\ x_{p1} \\ y_{p1} \\ \vdots \\ x_{pq} \\ y_{pq} \\ z_p
\end{bmatrix}
=
\begin{bmatrix}
f_{11} \\ g_{11} \\ \vdots \\ f_{1q} \\ g_{1q} \\ h_1 \\ \vdots \\ f_{p1} \\ g_{p1} \\ \vdots \\ f_{pq} \\ g_{pq} \\ h_p
\end{bmatrix} \quad (7)$$

5. Form right-hand sides parallelly for inner interface nodes:

$$\widetilde{g}_{ij} = g_{ij} - B_{ij}^T A_{ij}^{-1} f_{ij} \quad (11)$$

6. Form right-hand sides parallelly for outer interface nodes:

$$\widetilde{h}_i = h_i - \sum_{j=1}^q P_{ij}^T A_{ij}^{-1} f_{ij} \quad (12)$$

Therefore the intermediate Schur complement is formed.

Eq. (7) is condensed by removing the inner internal nodes through Gaussian elimination into a compact form, resulting in the intermediate Schur complement matrix, ready for further computations:

$$\begin{bmatrix}
\widetilde{D}_{11} & \cdots & E_{11,1q} & \widetilde{Q}_{11} & \cdots & O & \cdots & O & O \\
\vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots & \vdots \\
E_{11,1q}^T & \cdots & \widetilde{D}_{1q} & \widetilde{Q}_{1q} & \cdots & O & \cdots & O & O \\
\widetilde{Q}_{11}^T & \cdots & \widetilde{Q}_{1q}^T & \widetilde{R}_1 & \cdots & O & \cdots & O & F_{1,p} \\
\vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots & \vdots \\
O & \cdots & O & O & \cdots & \widetilde{D}_{p1} & \cdots & E_{p1,pq} & \widetilde{Q}_{p1} \\
\vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots & \vdots \\
O & \cdots & O & O & \cdots & E_{p1,pq}^T & \cdots & \widetilde{D}_{pq} & \widetilde{Q}_{pq} \\
O & \cdots & O & F_{1,p}^T & \cdots & \widetilde{Q}_{p1}^T & \cdots & \widetilde{Q}_{pq}^T & \widetilde{R}_p
\end{bmatrix}
\begin{bmatrix}
y_{11} \\ \vdots \\ y_{1q} \\ z_1 \\ \vdots \\ y_{p1} \\ \vdots \\ y_{pq} \\ z_p
\end{bmatrix}
=
\begin{bmatrix}
\widetilde{g}_{11} \\ \vdots \\ \widetilde{g}_{1q} \\ \widetilde{h}_1 \\ \vdots \\ \widetilde{g}_{p1} \\ \vdots \\ \widetilde{g}_{pq} \\ h_p
\end{bmatrix} \quad (13)$$

Then, eliminate inner interface nodes.

1. Extract submatrices $\widetilde{U}_i$, $\widetilde{V}_i$ and right-hand sides $\widetilde{g}_i$, $\widetilde{h}_i$ parallelly for inner and outer interface nodes in Eq. (13) within each outer subdomain:

$$\widetilde{U}_i = \begin{bmatrix} \widetilde{D}_{i1} & E_{i1,i2} & \cdots & E_{i1,iq} \\ E_{i1,i2}^T & \widetilde{D}_{i2} & \cdots & E_{i2,iq} \\ \vdots & \vdots & \ddots & \vdots \\ E_{i1,iq}^T & E_{i2,iq}^T & \cdots & \widetilde{D}_{iq} \end{bmatrix} \quad (14)$$

$$\widetilde{V}_i = [\widetilde{Q}_{i1}^T \quad \widetilde{Q}_{i2}^T \quad \cdots \quad \widetilde{Q}_{iq}^T]^T, \quad \widetilde{g}_i = [\widetilde{g}_{i1}^T \quad \widetilde{g}_{i2}^T \quad \cdots \quad \widetilde{g}_{iq}^T]^T \quad (15)$$

2. Form matrices parallelly for outer interface nodes:

$$\widetilde{R}_i = \widetilde{R}_i - \widetilde{V}_i^T \widetilde{U}_i^{-1} \widetilde{V}_i \quad (16)$$

3. Form right-hand sides parallelly for outer interface nodes:

$$\widetilde{h}_i = \widetilde{h}_i - \widetilde{V}_i^T \widetilde{U}_i^{-1} \widetilde{g}_i \quad (17)$$

And form the global Schur complement. The process of eliminating the inner internal and interface nodes employs the proposed parallel Schur complement computation method, ultimately resulting in the Schur complement for the selected ports (i.e., the outer interface nodes):

$$\begin{bmatrix}
\widetilde{R}_1 & F_{1,2} & \cdots & F_{1,p} \\
F_{1,2}^T & \widetilde{R}_2 & \cdots & F_{2,p} \\
\vdots & \vdots & \ddots & \vdots \\
F_{1,p}^T & F_{2,p}^T & \cdots & \widetilde{R}_p
\end{bmatrix}
\begin{bmatrix}
z_1 \\ z_2 \\ \vdots \\ z_p
\end{bmatrix}
=
\begin{bmatrix}
\widetilde{h}_1 \\ \widetilde{h}_2 \\ \vdots \\ \widetilde{h}_p
\end{bmatrix} \quad (18)$$

Last step is solution process. The solution to Eq. (7) begins with Eq. (18) to determine the outer interface unknowns z_i . Next, the subsequent equations are used to find the inner interface unknowns y_{ij} , and finally, solving the second equation below yields the inner internal unknowns x_{ij} :

$$\begin{bmatrix}
\widetilde{D}_{i1} & E_{i1,i2} & \cdots & E_{i1,iq} \\
E_{i1,i2}^T & \widetilde{D}_{i2} & \cdots & E_{i2,iq} \\
\vdots & \vdots & \ddots & \vdots \\
E_{i1,iq}^T & E_{i2,iq}^T & \cdots & \widetilde{D}_{iq}
\end{bmatrix}
\begin{bmatrix}
y_{i1} \\ y_{i2} \\ \vdots \\ y_{iq}
\end{bmatrix}
=
\begin{bmatrix}
\widetilde{g}_{i1} - \widetilde{Q}_{i1} z_i \\ \widetilde{g}_{i2} - \widetilde{Q}_{i2} z_i \\ \vdots \\ \widetilde{g}_{iq} - \widetilde{Q}_{iq} z_i
\end{bmatrix} \quad (19)$$

$$A_{ij} x_{ij} = f_{ij} - B_{ij} y_{ij} - P_{ij} z_i \quad (20)$$

Combining the above derivations, the Laplacian matrix $Gv = b$ is solved with nested DDM as in Algorithm 1.

C. The Overall Framework

Building on the optimizations outlined above, the overall framework of our proposed solver is presented in Algorithm 1, which consists of three stages:

- First, the PDN is divided into p outer subdomains, and then each outer subdomain is further subdivided into q inner subdomains, creating local grids labeled as $11 \sim pq$, where $m = p \times q$. This hierarchical strategy significantly reduces the size of the top-level problem by introducing an intermediate Schur complement, which, while still dense, is more suitable for parallel computation.

- Next, the local grids $i1 \sim iq$ within each outer subdomain i are used to construct the middle grid i through the computation of the Schur complement, which consolidates the conductance of most nodes onto specific nodes. The p middle grids are then combined through further node elimination to form the top grid. The solution process starts with solving the top grid, and its solution is then used to solve the p middle grids.
- Finally, the solutions of the middle grids are back-substituted into their respective local grid, as illustrated in Fig. 3.

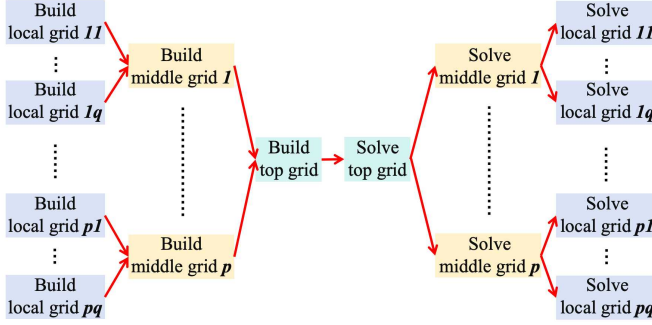


Fig. 3: Flow of the proposed nested DDM approach.

Efficient Elimination. In the local layer, instead of solving Eq. (8)-(12) individually, we concatenate A_{ij} , B_{ij} , D_{ij} , P_{ij} , Q_{ij} , and R_i into a unified matrix. This approach enables only a single factorization to yield the result, as shown in Eq. (2). To eliminate submatrix A_{ij} , techniques such as matrix reordering can be employed to reorganize the A_{ij} , thereby reducing fill-ins during factorization. Matrix reordering techniques can also be used in the middle layer when solving Eq. (16)-(17).

Maximizing Multithreading Efficiency. For the inner subdomains, each thread processes an individual submatrix. In the intermediate layer, only p tasks can be evidently parallelized, as described in Eq. (16)–(17) and (19). Therefore, q threads are used to solve each task. For the top layer, however, there are no obvious parallelization opportunities, as there is only one task, as indicated in Eq. (18). In this case, $p \times q$ threads are used to solve the single task.

IV. EXPERIMENTAL RESULTS

Our proposed framework, implemented in C++ and compiled with GCC 9.4.0 using the -O3 optimization flag, utilizes METIS [15] for graph partitioning and CHOLMOD direct solver [5] with Intel MKL [19] for matrix factorization. We test on the IBMPG [20] and THUPG [21] benchmarks and compare our method with three state-of-the-art solvers, the CHOLMOD-based serial solver, the CKTSO parallel solver [6], and the conventional DDM parallel solver. Although CHOLMOD supports multithreading, our experiments show no significant performance improvement in the parallel version, so we compare the serial version's execution time. All experiments are conducted on a Linux server with two Intel Xeon Silver 4210R CPUs and 128GB of RAM.

Algorithm 1 Proposed parallel solver with nested DDM

Input: Initial matrix G , number of outer subdomains p , number of inner subdomains q .

Output: Solution x_{ij} , y_{ij} , and z_i of Eq. (7).

- 1: Perform initial graph partitioning on G to generate p outer subdomains, resulting in outer interface nodes and outer internal nodes.
- 2: **for** each of the p outer subdomains in parallel **do**
- 3: Partition the outer internal nodes into q inner subdomains, resulting in inner interface nodes and inner internal nodes.
- 4: **end for**
- 5: **for** each of the $p \times q$ inner subdomains in parallel **do**
- 6: Build the local grid using inner internal nodes, inner interface nodes, and corresponding outer interface nodes.
- 7: Eliminate inner internal nodes via Eq. (8)-(12).
- 8: **end for**
- 9: **for** each of the p outer subdomains in parallel **do**
- 10: Build the middle grid, resulting in Eq. (13).
- 11: Eliminate inner interface nodes using Eq. (16)-(17).
- 12: **end for**
- 13: Build the global grid, resulting in Eq. (18).
- 14: Solve the global grid for z_i using Eq. (18).
- 15: **for** each of the p outer subdomains in parallel **do**
- 16: Solve the middle grid for y_{ij} using Eq. (19).
- 17: **end for**
- 18: **for** each of the $p \times q$ inner subdomains in parallel **do**
- 19: Solve the local grid for x_{ij} using Eq. (20).
- 20: **end for**

A. Results on DC Analysis of Power Grids

Table II provides the comparison results between the proposed nested DDM parallel solver and three other solvers. For DDM, the circuit matrix is divided into $p \times q$ subdomains, while nested DDM divides it into p outer subdomains, each with q inner subdomains. Optimal performance is observed with the divisions 2×2 , 2×4 , 2×8 , and 4×8 , which are used in Table II. Here, T_{loc} , T_{mid} , T_{top} and T represent the computation time for inner subdomains, intermediate Schur complement, global Schur complement and overall execution, respectively. T_{mt} is the total time for the intermediate and global Schur complements. The speedup ratios Sp_1 , Sp_2 , and Sp_3 indicate performance improvements over the CHOLMOD, CKTSO, and conventional DDM parallel solver, respectively.

It can be seen from Table II that our framework achieves an average 8.36 $\times$ speedup compared to the CHOLMOD-

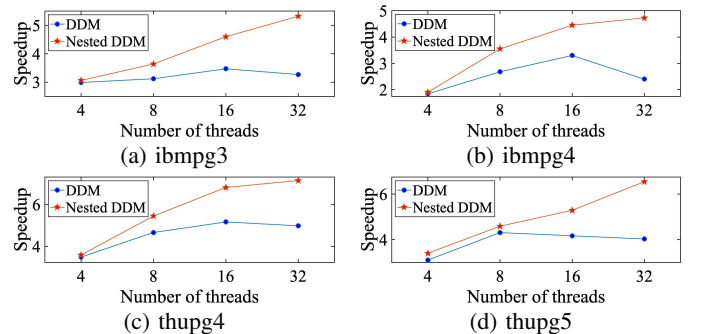


Fig. 4: Speedup comparison of DDM and nested DDM parallel solver over CHOLMOD-based serial solver.

TABLE II: Runtime Comparison of the Proposed Nested DDM Solver with CHOLMOD-based Serial Solver [5], CKTSO Parallel Solver [6], and Conventional DDM Solver.

Case	N	Serial T	CKTSO T	DDM			Proposed nested DDM					$Sp1$	$Sp2$	$Sp3$
				T_{loc}	T_{top}	T	T_{loc}	T_{mid}	T_{top}	T_{mt}	T			
ibmpg8	4	22.7	28.1	4.03	0.40	4.43	3.89	0.22	0.01	0.23	4.12	5.51×	6.82×	1.08×
	8		22.6	2.53	1.65	4.18	2.47	0.65	0.01	0.66	3.13	7.25×	7.22×	1.34×
	16		21.9	1.88	3.62	5.50	1.80	1.46	0.03	1.49	3.29	6.90×	6.66×	1.67×
	32		23.2	1.38	2.95	4.33	1.45	0.80	0.36	1.16	2.61	8.70×	8.89×	1.66×
thupg1	4	101	148	24.1	3.60	27.7	23.9	2.23	0.44	2.67	26.6	3.80×	5.56×	1.04×
	8		132	8.16	7.96	16.1	8.56	4.41	0.45	4.86	13.4	7.54×	9.85×	1.20×
	16		124	5.32	13.9	19.2	5.32	6.88	0.37	7.25	12.6	8.02×	9.84×	1.52×
	32		118	3.89	14.6	18.5	3.70	2.96	2.76	5.72	9.42	10.7×	12.5×	1.96×
thupg2	4	224	348	53.1	6.90	60.0	52.7	4.76	0.79	5.55	58.3	3.84×	5.97×	1.03×
	8		279	25.3	15.7	41.0	25.7	8.66	1.22	9.88	35.6	6.29×	7.84×	1.15×
	16		260	12.5	22.3	34.8	12.9	13.5	0.70	14.2	27.1	8.27×	9.59×	1.28×
	32		251	7.95	25.6	33.6	8.10	11.6	5.79	17.4	25.5	8.78×	9.84×	1.32×
thupg3	4	246	460	63.2	9.60	72.8	61.7	6.38	0.98	7.35	69.1	3.56×	6.66×	1.05×
	8		355	41.8	21.1	62.9	39.9	13.8	0.80	14.6	54.5	4.51×	6.51×	1.15×
	16		337	17.7	30.0	47.7	17.7	18.0	0.81	18.8	36.5	6.74×	9.23×	1.31×
	32		335	11.2	29.6	40.8	11.1	13.4	5.26	18.7	29.8	8.26×	11.2×	1.37×
thupg4	4	360	687	91.2	11.3	103	88.6	9.82	2.02	11.8	100	3.60×	6.87×	1.03×
	8		563	51.2	25.9	77.1	51.1	13.5	1.47	15.0	66.1	5.45×	8.52×	1.17×
	16		525	23.5	46.1	69.6	22.8	28.3	1.67	30.0	52.8	6.81×	9.94×	1.32×
	32		484	16.5	55.7	72.2	16.6	22.6	11.2	33.8	50.4	7.14×	9.60×	1.43×
thupg5	4	496	1100	129	31.0	160	128	15.1	2.60	17.7	146	3.40×	7.53×	1.10×
	8		861	72.8	42.1	115	72.3	33.0	2.66	35.7	108	4.59×	7.97×	1.06×
	16		776	45.3	73.2	119	46.9	44.8	2.00	46.8	93.7	5.29×	8.28×	1.27×
	32		727	23.4	100	123	22.8	33.7	19.2	52.9	75.7	6.55×	9.60×	1.62×
Average	-	-	-	-	-	-	-	-	-	-	-	-	8.44×	1.30×

based serial solver utilizing 32 threads, due to multi-level domain decomposition that enhances parallelism by breaking the problem into smaller independent subproblems. Compared to the CKTSO parallel solver, it achieves an average speedup of 8.44× in total runtime, owing to the more structured and sparsely connected nature of power grids. In comparison to the DDM parallel solver, our approach achieves a 1.70× speedup in solving the global Schur complement. This is due to the introduction of an intermediate Schur complement, which enables additional parallelism by redistributing top-level task to the intermediate layer in nested DDM. The improvement is evident when comparing T_{top} in DDM with T_{mt} in nested DDM. As a result, our method provides a 1.30× total runtime speedup over the DDM parallel solver.

To highlight the superior scalability of our framework compared to conventional DDM, we compare the two parallel solvers. As shown in Fig. 4, the nested DDM approach exhibits significantly better scalability with an increasing number of threads, while DDM's performance plateaus early due to inefficiencies in handling the large, dense global Schur complement.

B. Evaluation of parallel Schur complement computation

We further demonstrate the effectiveness of our proposed parallel method for computing the Schur complement using two power grid cases. In these examples, the red nodes represent the selected ports predicted to experience the highest worst-case noise, for which the Schur complement must be computed. The remaining nodes are eliminated to produce an equivalent conductance matrix for the selected ports (i.e., the key points we need to focus on). As shown in Fig. 5, our partition-based parallel computation method achieves significant performance improvements, with speedups of 6.73× and 11.6× when using 32 threads. The method is applicable to PDNs with various grid shapes, and its parallel efficiency depends on the initial partitioning. Fewer interface nodes (e.g.,

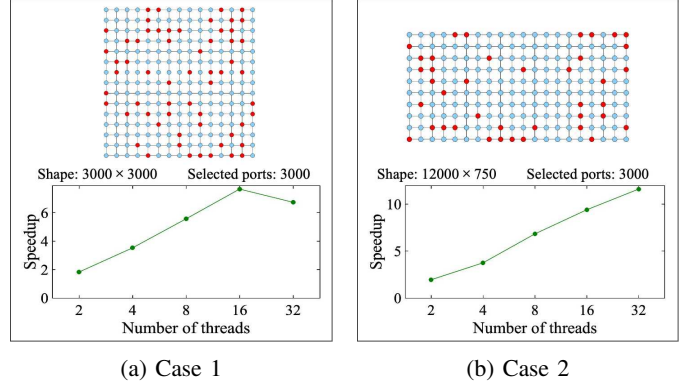


Fig. 5: Comparison of speedup in DDM for Schur complement computation across different grid shapes.

750 × 31 in Case 2 compared to 3000 × 10 in Case 1 for 32 threads) generally result in higher parallel efficiency.

V. CONCLUSION

In this paper, we have presented an efficient parallel nested domain decomposition solver for power grid analysis, aimed at enhancing the scalability of the traditional DDM parallel solver. By introducing a parallel method for computing the Schur complement and integrating it with conventional DDM, we effectively addresses the challenges of managing the large, dense global Schur complement and the computational complexity inherent in large-scale PDNs. Our results demonstrate that the proposed method not only accelerates the solution process but also optimizes resource utilization, making it highly suitable for modern IC design.

ACKNOWLEDGEMENT

This work was supported by NSFC (Grant No. 62034007), Zhejiang Provincial NSF (Grant No. D24F040002), NSFC (Grant No. 62204265, 92473107, U23A20301 and 62234010).

REFERENCES

- [1] E. Chiprout, "On-die power grids: The missing link," in *Proceedings of the 47th Design Automation Conference*, 2010, pp. 940–945.
- [2] C. Zhuo, G. Wilke, R. Chakraborty, A. A. Aydin, S. Chakravarty, and W.-K. Shih, "Silicon-validated power delivery modeling and analysis on a 32-nm ddr i/o interface," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 23, no. 9, pp. 1760–1771, 2014.
- [3] C. Zhuo, K. Unda, Y. Shi, and W.-K. Shih, "From layout to system: Early stage power delivery and architecture co-exploration," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 7, pp. 1291–1304, 2018.
- [4] C. Wen, Y. Cai, and C. Zhuo, "Impact of supply noise on nano-meter vlsi design: Hard or soft threshold?" in *2021 China Semiconductor Technology International Conference (CSTIC)*. IEEE, 2021, pp. 1–4.
- [5] Y. Chen, T. A. Davis, W. W. Hager, and S. Rajamanickam, "Algorithm 887: Cholmod, supernodal sparse cholesky factorization and update/downdate," *ACM Transactions on Mathematical Software (TOMS)*, vol. 35, no. 3, pp. 1–14, 2008.
- [6] X. Chen, "Numerically-stable and highly-scalable parallel lu factorization for circuit simulation," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 2022, pp. 1–9.
- [7] R. Barrett, M. Berry, T. F. Chan, J. Demmel, J. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine, and H. Van der Vorst, *Templates for the solution of linear systems: building blocks for iterative methods*. SIAM, 1994.
- [8] C. Zhuo, J. Hu, M. Zhao, and K. Chen, "Power grid analysis and optimization using algebraic multigrid," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 4, pp. 738–751, 2008.
- [9] Z. Feng, "Spectral graph sparsification in nearly-linear time leveraging efficient spectral perturbation analysis," in *Proceedings of the 53rd Annual Design Automation Conference*, 2016, pp. 1–6.
- [10] Z. Liu, W. Yu, and Z. Feng, "fegrass: Fast and effective graph spectral sparsification for scalable power grid analysis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 3, pp. 681–694, 2021.
- [11] Q. Zhou, K. Sun, K. Mohanram, and D. C. Sorensen, "Large power grid analysis using domain decomposition," in *Proceedings of the Design Automation & Test in Europe Conference*, vol. 1. IEEE, 2006, pp. 1–6.
- [12] K. Sun, Q. Zhou, K. Mohanram, and D. C. Sorensen, "Parallel domain decomposition for simulation of large-scale power grids," in *2007 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 2007, pp. 54–59.
- [13] Z. Liu and W. Yu, "pgrass-solver: A graph spectral sparsification-based parallel iterative solver for large-scale power grid analysis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 9, pp. 3031–3044, 2023.
- [14] H. Zhou, D. Yang, Y. Lin, Y. Dai, and Q. Chen, "A parallel acceleration technique based on bordered block diagonal matrix reordering for exponential integrator method," in *2024 2nd International Symposium of Electronics Design Automation (ISED)*. IEEE, 2024, pp. 94–99.
- [15] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.
- [16] C.-H. Pao, A.-Y. Su, and Y.-M. Lee, "Xgbir: An xgboost-based ir drop predictor for power delivery network," in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2020, pp. 1307–1310.
- [17] V. A. Chhabria, V. Ahuja, A. Prabhu, N. Patil, P. Jain, and S. S. Sapatnekar, "Thermal and ir drop analysis using convolutional encoder-decoder networks," in *Proceedings of the 26th Asia and South Pacific Design Automation Conference*, 2021, pp. 690–696.
- [18] X. Dong, Y. Chen, X. Yin, and C. Zhuo, "Worst-case dynamic power distribution network noise prediction using convolutional neural network," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 1225–1230.
- [19] I. Corporation. (2024) Intel oneapi math kernel library. Online. Accessed: 2024. [Online]. Available: <https://software.intel.com/content/www/us/en/develop/tools/oneapi/components/onemkl.html>
- [20] S. R. Nassif. (2020) Ibm power grid benchmarks. Online. Accessed: 2024. [Online]. Available: <https://web.ece.ucsb.edu/lip/PGBenchmarks/ibmpgbench.html>
- [21] J. Yang and Z. Li. (2012) Thu power grid benchmarks. Online. Accessed: 2024. [Online]. Available: <http://tiger.cs.tsinghua.edu.cn/PGBench/>